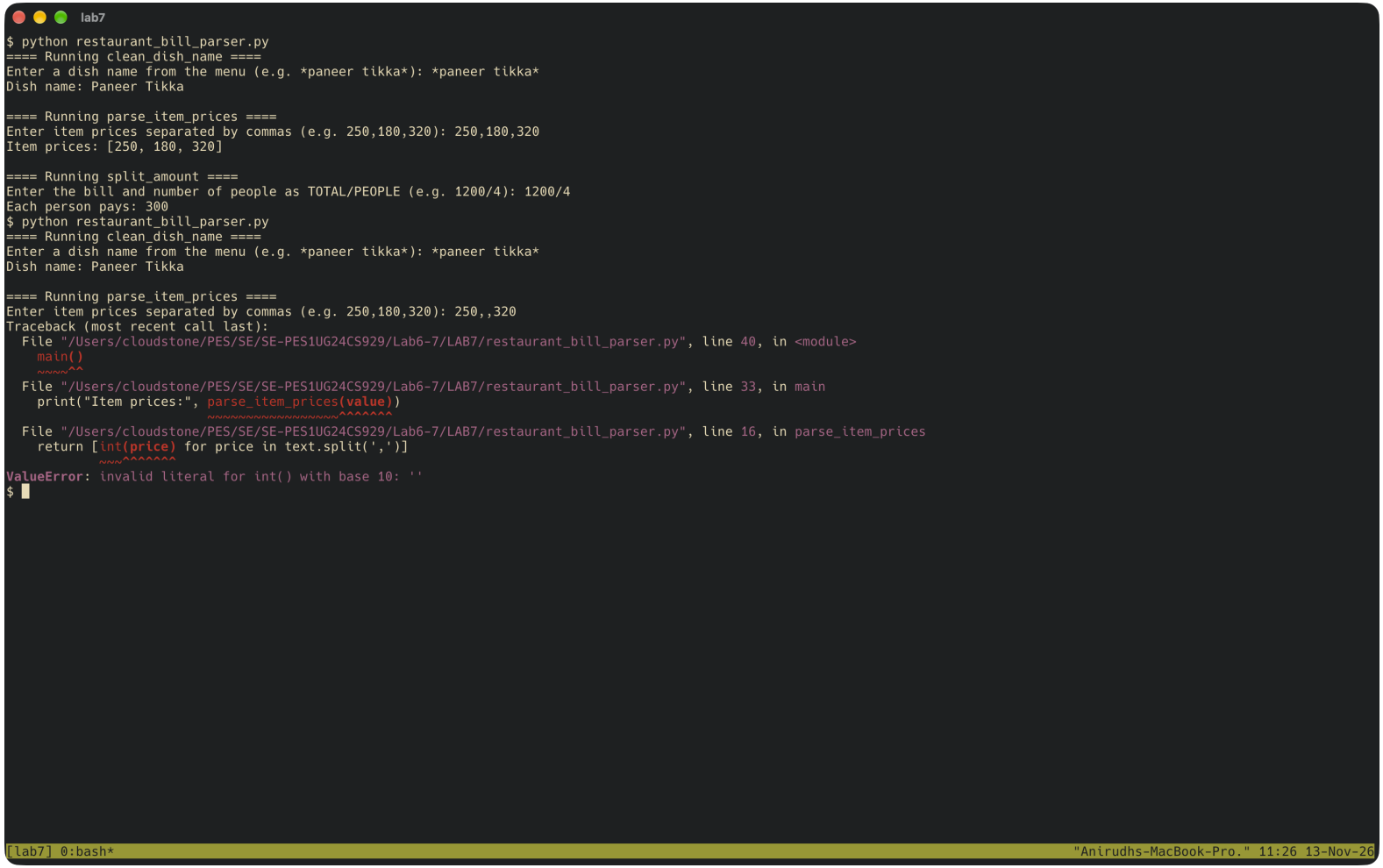
Lab 7: Fuzzing for Input Validation Bugs

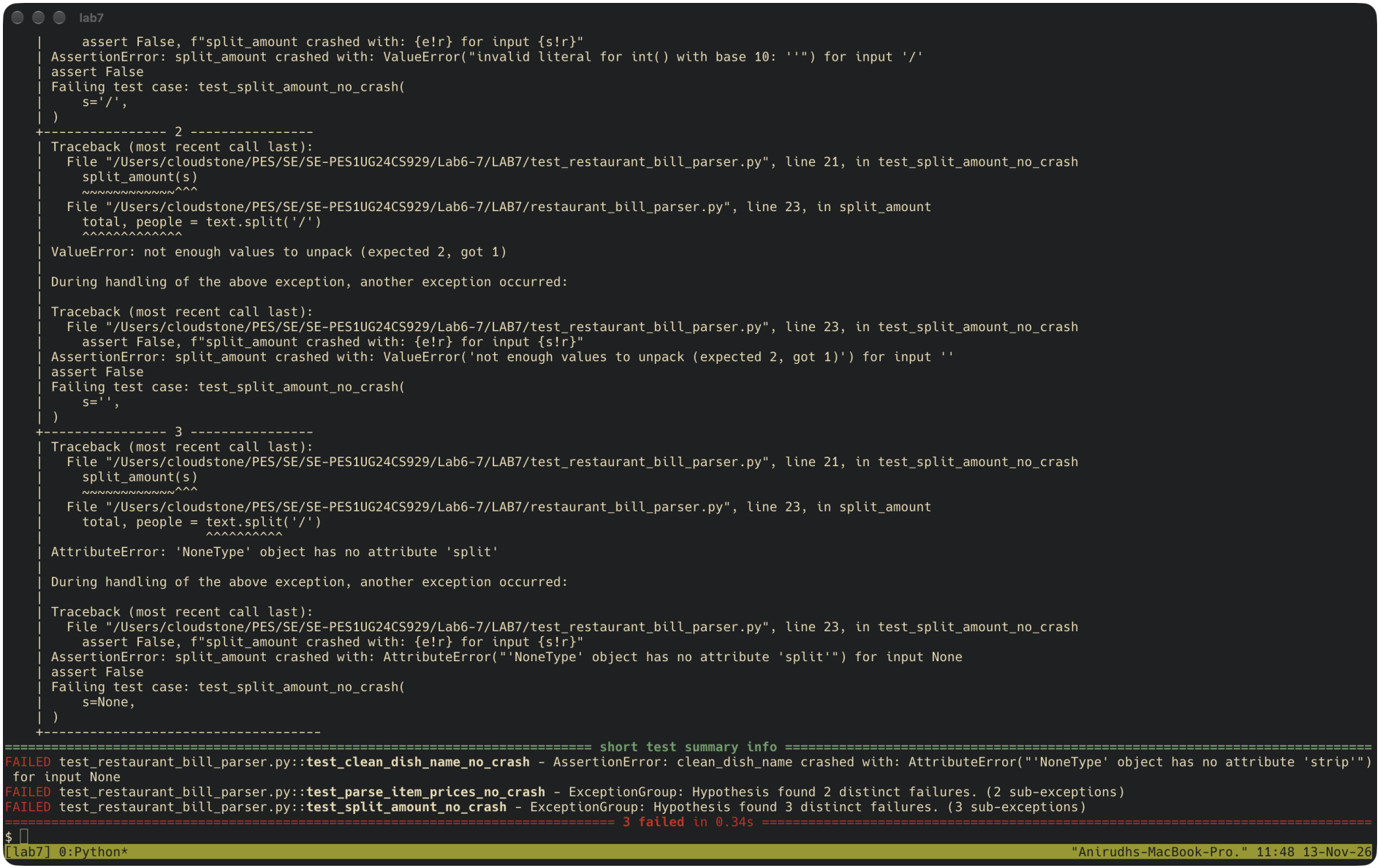
Name: Anirudh | SRN: PES1UG24CS929 | Module: `restaurant_bill_parser.py` (L6_L7_58 Restaurant Bill) | Tool: Hypothesis + pytest

SS1: `python restaurant_bill_parser.py` on the buggy module



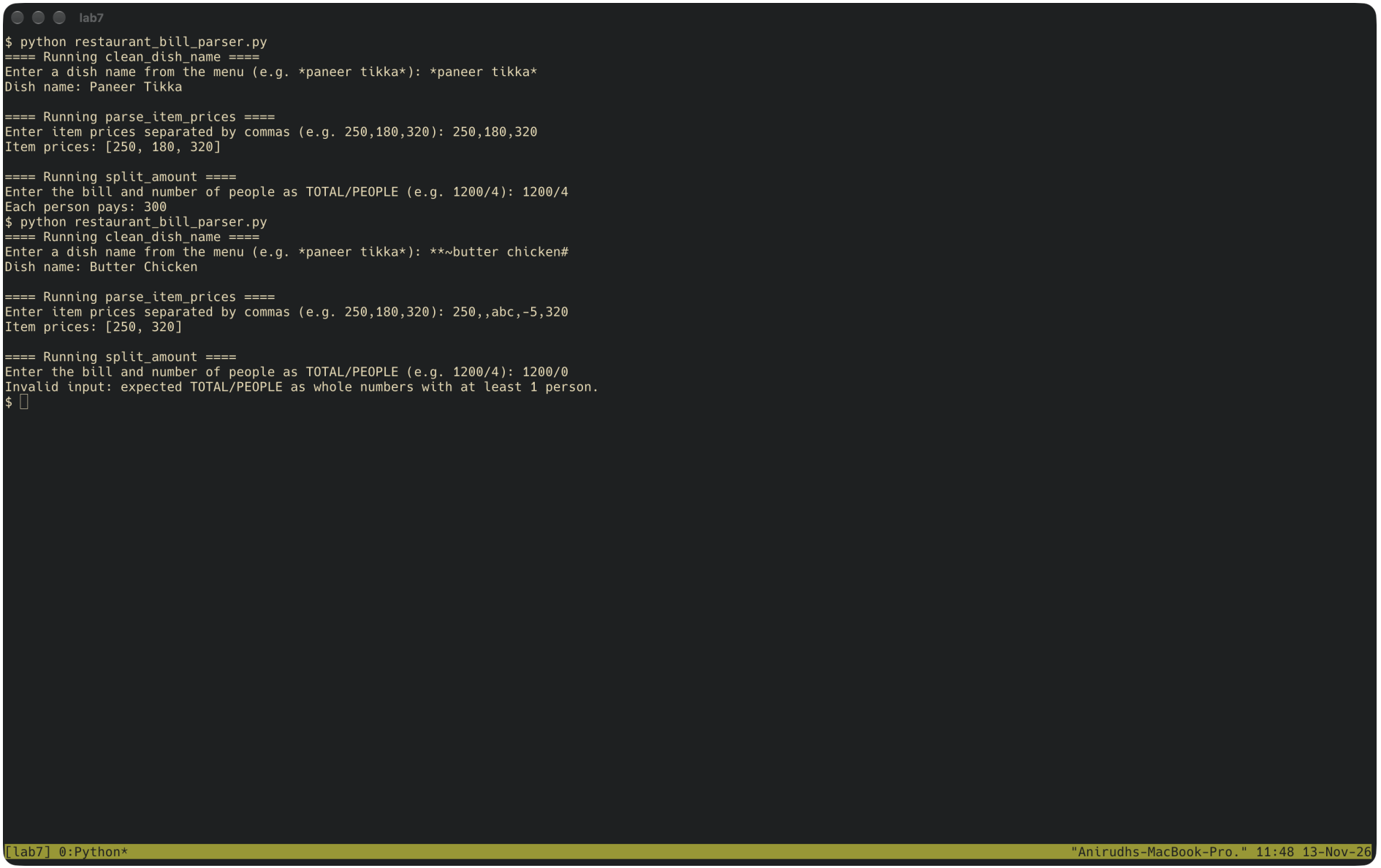
Run 1 with well-formed input works. Run 2 enters `250,,320` (an empty price) and the program crashes with an uncaught `ValueError` inside `parse_item_prices`.

SS2: `python -m pytest test_restaurant_bill_parser.py` **on the buggy module**



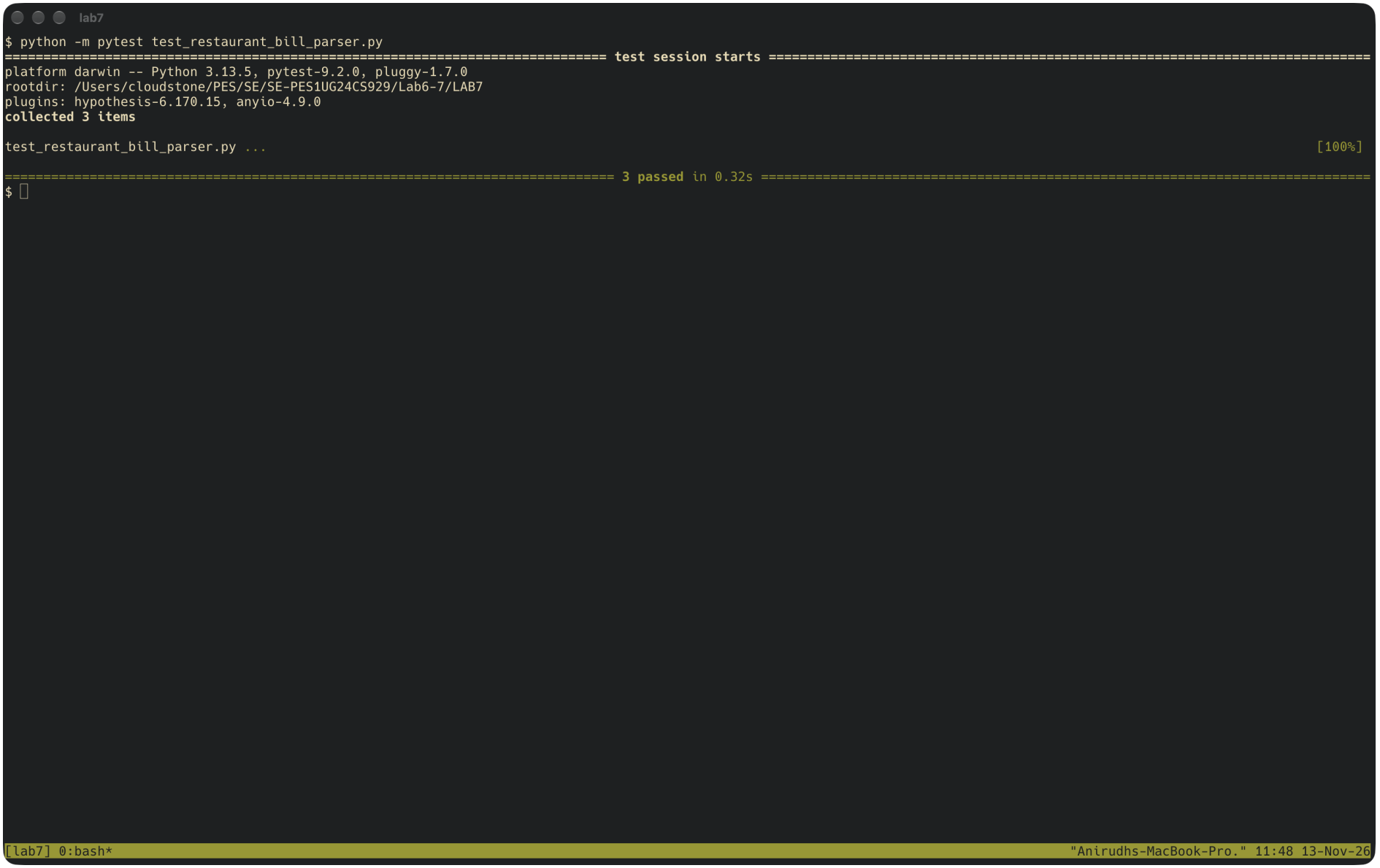
All 3 Hypothesis tests fail (**3 failed**). Hypothesis shrinks each failure to a minimal input such as `None`, `''` and `'/'`.

SS3: `python restaurant_bill_parser.py` **on the fixed module**



The same normal input still gives the same answers. The inputs that crashed before (empty, non-numeric and negative prices, a split by zero) are now handled without a traceback.

SS4: `python -m pytest test_restaurant_bill_parser.py` **on the fixed module**



All 3 Hypothesis tests pass (**3 passed**) against the fixed module.

Bugs found by Hypothesis (SS2) and how they were fixed

Function	Falsifying input	Exception raised on the buggy code	Cause
`clean_dish_name`	`None`	`AttributeError: 'NoneType' object has no attribute 'strip'`	Assumed a string was always passed in.
`parse_item_prices`	`''`	`ValueError: invalid literal for int() with base 10: ''`	Empty element: ''.split(',') gives [''], and int('') fails.
`parse_item_prices`	`None`	`AttributeError: 'NoneType' object has no attribute 'split'`	No type check before calling .split().
`split_amount`	`'/'`	`ValueError: invalid literal for int() with base 10: ''`	Both sides of the slash are empty.
`split_amount`	`''`	`ValueError: not enough values to unpack (expected 2, got 1)`	Tuple-unpacking assumed exactly one '/'.
`split_amount`	`None`	`AttributeError: 'NoneType' object has no attribute 'split'`	No type check before calling .split().

The same unchecked assumptions also allow other crashes that the fix covers: a zero divisor (`1200/0` raises `ZeroDivisionError`), too many slashes (`1/2/3`), non-numeric text (`abc`), and numbers that `int()` accepts but are not valid prices or counts (negative values, `1_000`, `+5`, Unicode digits). Also, `clean_dish_name` stripped whitespace before removing the markers, so `'* paneer *'` kept stray spaces; it now strips afterwards.

What the fixed module does

• **clean_dish_name**: returns `''` for a non-string, otherwise removes the markers, strips, and title-cases.

• **parse_item_prices**: returns `[]` for a non-string. Each comma-separated entry is accepted only if it is plain ASCII digits (`[0-9]+`); empty, non-numeric, negative or decimal entries are skipped.

• **split_amount**: returns `None` (instead of raising) for a non-string, anything other than exactly two whole numbers around one '/', or zero people. `main()` prints a clear message in that case.

• A shared helper converts a token to an int and also catches the `ValueError` Python raises for extremely long digit strings.

Reflection

1. What kinds of bugs did fuzzing find that you might not have tested by hand?

Hypothesis immediately produced inputs I would not normally type: `None`, the empty string, a lone `/`, and entries that are empty only because of a stray comma. The code assumed well-formed input (the docstrings even said so), and each assumption turned into an uncaught exception. Hypothesis also shrinks a failure to the smallest input that triggers it, which made the cause of each crash obvious.

2. How did you fix them?

By validating at the boundary instead of trusting the caller: check the type first, split, and convert each piece with a strict helper that returns `None` when a token is not a whole number. Invalid entries are skipped (`parse_item_prices`) or reported as `None` (`split_amount`), the zero divisor is checked explicitly, and the helper also guards against the `ValueError` that `int()` raises for huge digit strings. I re-ran the tests (SS4) and a larger fuzz run of 20,000 mixed inputs to confirm there were no crashes.

3. Why is fuzzing / property-based testing valuable compared with ordinary unit tests, and what are its limits?

A unit test only checks the cases its author thought of; a property such as 'this function never raises' is checked against hundreds of generated inputs, including ones nobody imagined. The limit is that a 'no crash' property says nothing about the result being correct (a function that always returns `None` would pass), so fuzzing complements example-based tests that assert exact outputs, and it only finds bugs in inputs the strategy can generate.